

Native E-Signatures for the EHR

What to do, and what not to do, when building legally binding signing into our custom EHR.

Read first

The signing mechanics are the easy part. What you are really building is **evidence**: if a signature is disputed years from now, the record alone must show who signed, what they saw, that they meant to sign, and that nothing changed afterwards. Every rule in this guide serves that goal. When in doubt, choose the option that produces better evidence.

1. Scope of v1

Version 1 covers signers who are **already authenticated in the EHR**: patients in the portal, patients on an in-clinic tablet, and clinicians signing off on notes and forms. Documents come from PDF templates we control. Target effort is three to five engineer-weeks.

Deliberately out of v1: remote signing by email link for people without an account, a drag-and-drop template builder, bulk send, and arbitrary uploaded PDFs. Each of these can double the project. Add them later, one at a time, only when there is a real need.

2. What makes a signature binding

In the US, the ESIGN Act and UETA make electronic signatures as valid as ink, provided four things can be shown. Each one maps directly to something you build.

Requirement	What it means	What you build
Intent and consent	The signer chose to sign, and agreed to do business electronically.	ESIGN disclosure, explicit agree step, deliberate signing action, stored consent version.
Attribution	The signature can be tied to a specific person.	Authenticated session, re-authentication at signing for clinicians, modelled guardians and witnesses.
Integrity	The document cannot be changed after signing without detection.	Server-side flattening, SHA-256 hashes, PAdES cryptographic seal with trusted timestamp.
Record retention	The record is kept, reproducible, and available to the signer.	Write-once storage, append-only audit trail, certificate of completion, signer copy.

3. The signing pipeline

Every document goes through the same steps, in this order. Each step writes an audit event.

#	Step	What happens
1	Prepare	Server fills a published, versioned template with chart data and renders the PDF. Hash the exact bytes that will be shown.
2	Present	Signer views the whole document in the app. Record that it was presented and viewed.
3	Consent	Signer accepts the electronic signature disclosure. Record which version of the text they accepted.
4	Authenticate	Confirm identity at the moment of signing. Clinicians re-enter a password, PIN, or MFA. Patients use their session, optionally plus a one-time code.
5	Capture	Signer draws, types, or clicks to sign each required field. The client sends only the signature input, never a PDF.
6	Finalize	Server places signatures, flattens all fields, and appends the certificate of completion generated from the audit trail.
7	Seal	Server applies a PAdES signature using a key held in KMS or an HSM, with an RFC 3161 timestamp. Hash the sealed bytes.
8	Store and deliver	Write the sealed PDF to write-once storage, link it to the chart, and give the signer a copy.
9	Verify	An internal tool re-validates any stored document on demand: seal intact, hashes match, audit chain unbroken.

4. Do's and don'ts

Scope and templates

DO	DON'T
Ship v1 for authenticated signers only, with a fixed set of templates.	Start with email-link signing or a visual template builder. They are the two largest time sinks in this domain.
Define fields in code, or as AcroForm fields inside PDFs we author.	Accept arbitrary uploaded PDFs. Rotated pages, encrypted files, scans, and existing signatures will eat weeks.
Version every template. A published version is immutable.	Edit a template in place once anyone has signed against it. Publish a new version instead.

Consent and intent

DO	DON'T
Show the ESIGN disclosure and require an explicit agree action before the first signature. Store the version of the text accepted.	Pre-tick the consent box, bury consent in the terms of service, or infer it from the user continuing.
Require a deliberate act for every signature on every document.	Silently re-apply a saved signature. A saved signature is fine only when the user actively chooses to apply it here.
Let the signer review the full document before the sign control becomes available, and offer a paper alternative.	Make signing reachable before the document has been presented, or make e-signing the only option.
Give the signer their signed copy straight away, by download and in the portal.	Keep the signed copy visible only to staff.

Identity and attribution

DO	DON'T
Bind each signature to a user ID and session. Re-authenticate clinicians at the moment of signing.	Rely on a long-lived session for clinician attestations, or tolerate shared logins anywhere near signing.
Model guardians, witnesses, interpreters, and proxies explicitly: who signed, in what capacity, for which patient.	Let staff sign for a patient by typing the patient's name, or record a guardian's signature as the patient's.
For in-clinic tablets, record which staff member started the session and how the patient's identity was checked.	Attribute a kiosk signature to the logged-in staff member's identity.

Document integrity and sealing

DO	DON'T
Build the final PDF on the server, flatten it, then seal it with PAdES (B-LT profile, RFC 3161 timestamp) using pyHanko, or pdf-lib with @signpdf.	Write your own cryptography, byte-range handling, or signature container. Use the library.
Keep the signing key in a cloud KMS or HSM and sign through the library's external-signer interface.	Put a .p12 or .pem private key in the repo, environment variables, a container image, or on an app server disk.
Hash the exact bytes shown to the signer and the final sealed bytes. Store both hashes in the audit trail.	Trust the client. Never accept a client-generated PDF, client-computed hash, or client-side flattening as the record.
For multiple signers, add each signature as an incremental update so earlier signatures stay valid.	Re-render, optimise, compress, or re-save a PDF after any signature has been applied. It breaks the seal.
Plan certificate rotation from day one and embed long-term validation data so old documents stay verifiable.	Assume a self-signed certificate earns Adobe's green tick. It is tamper-evident but shows as validity unknown. An Adobe-trusted certificate is a purchasing decision; raise it with the team lead.

Audit trail

DO	DON'T
Write an append-only event log covering created, presented, viewed, consented, authenticated, signed, declined, voided, sealed, and downloaded.	Store audit events anywhere the application role can UPDATE or DELETE. Grant INSERT and SELECT only.
Chain events: each row stores the hash of the previous row. Use server time in UTC.	Use client-supplied timestamps, or plan to rebuild the trail from application logs after the fact.
Generate the certificate of completion from the log and append it before sealing.	Put more PHI in the certificate than is needed. It travels with the document wherever it goes.

Storage and retention

DO	DON'T
Store sealed PDFs encrypted in write-once storage such as S3 Object Lock, linked to the chart.	Overwrite or delete a signed document. A correction is a new document that voids and references the old one.
Apply the medical record retention schedule. Ask compliance for the figures; they vary by state and are longer for minors.	Let generic cleanup jobs, TTLs, or account-deletion flows touch signed records.

Security and PHI

DO	DON'T
Treat signing like any PHI system: authorisation on every document fetch, access logging, encryption in transit and at rest.	Put PHI or guessable document IDs in URLs, emails, text messages, logs, error trackers, or analytics.
Use single-use, short-lived tokens bound to one signer and one document for any signing link.	Email the document as an attachment. Send a link that requires authentication.
Rate-limit and alert on the signing and re-authentication endpoints.	Send documents to any third party without a BAA, including PDF conversion and preview services. A timestamp authority only receives a hash, which is fine.

User experience

DO	DON'T
Make the flow work on phones and tablets, with keyboard and screen reader support. Offer typed and click-to-sign as well as drawn signatures.	Force a drawn signature. It excludes people and adds no legal weight; the audit trail is what carries the weight.
Show clear state: what is being signed, how many fields remain, and a confirmation with the signed copy.	Allow double submission. Signing must be idempotent, and a retried request must not create a second signature.

Testing and failure handling

DO	DON'T
Add an automated test that flips one byte in a sealed PDF and asserts that verification fails. Also open real outputs in Adobe Acrobat and check the signature panel.	Verify only with the same library that produced the signature.
Test the unhappy paths: decline, void, expiry, session timeout mid-signing, second signer after first, and KMS or timestamp outage.	Fail open. If sealing cannot complete, hold the document as pending and retry. Never show a document as complete until it is sealed and stored.

Process and licensing

DO	DON'T
Have counsel review the consent wording, the certificate of completion, and the list of document types before launch.	Decide for yourself which document types may be signed electronically.
Write a short internal note on how our signatures work and how to verify one. If a signature is challenged, this is what the team will reach for.	Leave verification knowledge in one engineer's head.
Read DocuSeal, Documenso, and OpenSign for ideas about flows and data models.	Copy code from them. All three are AGPL, and copied code would put our codebase under the same licence.

5. Stop and escalate

These are not engineering decisions. If a request touches any of them, stop and raise it before writing code.

Prescriptions	Electronic prescribing of controlled substances is governed by DEA rules that require a certified application, identity proofing, and two-factor signing. We integrate a certified e-prescribing vendor. We never build this.
Witness or notary	Some documents, such as certain advance directives and consents, need a witness or notary under state law. A basic e-signature may not be enough. Counsel decides, per document type and per state.
Non-US signers	Outside the US other regimes apply. In the EU, eIDAS may require an advanced or qualified signature, which needs a trust service provider.
New doc types	Any document type not on the approved list goes to compliance first, even if the template is trivial to add.

6. Audit event: minimum fields

Field	Notes
event_id, sequence	Monotonic per document. Gaps are a red flag during verification.
document_id, template_id, template_version	Ties the event to the exact template that was used.
event_type	created, presented, viewed, consented, authenticated, signed, declined, voided, sealed, downloaded.
actor_user_id, actor_role, capacity	Capacity covers self, guardian, witness, interpreter, clinician. Include on_behalf_of_patient_id where relevant.
auth_method	Session, password re-entry, PIN, MFA, one-time code, or staff-verified in clinic.
occurred_at	Server clock, UTC. Never from the client.
ip_address, user_agent	Captured server-side from the request.
document_sha256	Hash of the presented bytes, or of the sealed bytes for the sealed event.
consent_text_version	Set on the consented event.
prev_event_hash, event_hash	Hash chain. Verification walks the chain from the first event to the last.

7. Suggested build order

Order	Piece	Rough effort
1	Audit trail table, hash chain, and event writer. Everything else depends on it.	3-4 days
2	Sealing spike: flatten, PAdES seal through KMS, timestamp, verify in Acrobat. This is the highest technical risk, so prove it early.	3-5 days
3	Template versioning and server-side PDF preparation.	2-3 days
4	Viewer and signature capture UI, mobile and accessible.	3-4 days
5	Consent step and re-authentication at signing.	2-3 days
6	Certificate of completion, write-once storage, chart link, signer copy.	3-4 days
7	Multi-signer state machine: ordering, decline, void, expiry. Skip if v1 is single-signer.	3-5 days
8	Verification tool, tamper tests, unhappy-path tests, security review.	3-5 days

8. Definition of done

- ☐ Consent disclosure is shown, explicitly accepted, and its version is stored for every signer.
- ☐ Clinician signatures require re-authentication at the moment of signing.
- ☐ Guardians, witnesses, and proxies are recorded in their own capacity.
- ☐ The final PDF is built and flattened on the server; the client never supplies document bytes or hashes.
- ☐ The sealed PDF validates in Adobe Acrobat, and a one-byte change makes validation fail.
- ☐ The signing key lives in KMS or an HSM, and a rotation procedure is written down.
- ☐ The application's database role cannot update or delete audit events, and the hash chain verifies end to end.
- ☐ Sealed documents sit in write-once storage and are excluded from cleanup and deletion jobs.
- ☐ The signer receives a copy immediately.
- ☐ No PHI appears in URLs, emails, text messages, logs, or error tracking.
- ☐ KMS and timestamp outages leave documents pending, never falsely complete, and sealing retries on recovery.
- ☐ Counsel has approved the consent wording, the certificate of completion, and the document type list.
- ☐ An internal note explains how signatures work and how to verify one.

This guide is engineering guidance, not legal advice. Requirements vary by state and by document type; counsel and compliance have the final say on what may be signed electronically and how long it must be kept.